



**PERIYAR
MANIAMMAI**
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University)
Established under Sec. 3 of UGC Act, 1956 - NAAC Accredited
think • innovate • transform

DEPARTMENT OF COMPUTER SCIENCE

ASSIGNMENT - I

NAME : HATHIM

REGISTER NUMBER : 124012173087

CLASS : B.Sc., ARTIFICIAL INTELLIGENCE

SEMESTER : V

COURSE NAME : NEURAL NETWORKS

COURSE CODE : U24AI303

**TOPIC : NEURAL NETWORK FUNDAMENTAL
AND ACTIVATION FUNCTION**

DATE OF SUBMISSION : 31/08/2026

1. Introduction to Neural Networks

A neural network is a computational model inspired by the way biological neurons process information. It is one of the fundamental technologies behind modern artificial intelligence and machine learning.

A neural network receives input data, processes that information through interconnected computational units called neurons, and produces an output. During training, the network adjusts numerical parameters called weights and biases so that its predictions become more accurate.

Why Neural Networks Are Important

Neural networks are useful because they can learn complex relationships directly from data. Traditional programs normally require explicit rules written by a programmer, whereas a neural network can learn useful patterns from examples.

They are widely used in image recognition, speech processing, natural language processing, recommendation systems, medical analysis, fraud detection, robotics and forecasting.

Basic Idea

The central idea is simple: input values are multiplied by weights, a bias is added, and the resulting value is passed through an activation function. Multiple neurons are combined into layers, creating a network capable of learning increasingly complex representations.

Typical Neural Network Flow

Input Data → Weighted Sum → Activation Function → Hidden Layers → Output

The quality of a neural network depends on its architecture, training method, data, parameters and activation functions.

2. Fundamental Components of a Neural Network

Neuron

A neuron is the basic computational unit of a neural network. It receives one or more inputs and combines them using learned weights and a bias.

Weights

A weight represents the importance assigned to an input. During training, weights are updated to reduce prediction error. A large positive weight strengthens an input's influence, while a negative weight can suppress it.

Bias

Bias is an additional learnable value that allows a neuron to shift its activation threshold. It helps the network represent functions that would otherwise be difficult to model.

Weighted Sum

For inputs $x_1, x_2, \dots, x_n$ and weights $w_1, w_2, \dots, w_n$, the neuron first calculates $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$.

Activation

The value z is then passed through an activation function f . The neuron's output is $y = f(z)$. This nonlinear transformation is essential for learning complex patterns.

Connections

Neurons are connected between layers. In a fully connected layer, every neuron in one layer can connect to every neuron in the next layer.

3. Neural Network Architecture

Input Layer

The input layer receives the features of a problem. For example, an image classifier may receive pixel values, while a student-performance model may receive attendance, marks and study hours.

Hidden Layers

Hidden layers perform intermediate computations. A network may have one hidden layer or many. Deep neural networks contain multiple hidden layers and can learn hierarchical representations.

Output Layer

The output layer produces the final prediction. Its structure depends on the task: one output may be used for regression, multiple outputs for classification, and probability distributions for multiclass prediction.

Shallow vs Deep Networks

A shallow network has relatively few hidden layers. A deep network contains several hidden layers and is capable of representing complicated patterns, although it generally requires more data, computation and careful training.

Feedforward Neural Network

In a feedforward network, information moves from the input layer toward the output without forming cycles. A multilayer perceptron (MLP) is a common feedforward architecture.

Other Architectures

Convolutional neural networks are commonly associated with images, recurrent and related sequence models handle sequential information, and transformers are widely used for language and other sequence-based tasks.

4. How Neural Networks Learn

Training Data

Training begins with examples containing inputs and, for supervised learning, expected outputs. The model uses these examples to learn parameters.

Forward Propagation

During forward propagation, input data travels through each layer. Every neuron computes its weighted sum, applies an activation function, and passes the result to the next layer.

Loss Function

The predicted output is compared with the target using a loss function. Mean squared error is common for regression, while cross-entropy losses are widely used for classification.

Backpropagation

Backpropagation calculates how much each parameter contributed to the prediction error by applying the chain rule of calculus. These gradients indicate how parameters should change.

Optimization

An optimizer updates the weights and biases using gradients. Gradient descent and its variants, such as Adam, are commonly used.

Epoch and Batch

An epoch represents one complete pass through the training dataset. A batch is a smaller group of examples processed together. Batch-based training makes learning practical for large datasets.

Generalization

A good neural network should perform well on unseen data. Overfitting occurs when a model memorizes training examples instead of learning patterns that generalize.

5. Activation Functions – Fundamentals

An activation function determines the output of a neuron after the weighted sum and bias are calculated. Activation functions are one of the most important components of neural networks.

Why Activation Functions Are Needed

Without a nonlinear activation function, stacking many linear layers would still produce an overall linear transformation. Such a network would have limited ability to model nonlinear relationships.

Mathematical Form

A neuron generally computes $z = \sum(w_i x_i) + b$ and then produces $a = f(z)$, where f is the activation function.

Nonlinearity

Nonlinear activation functions allow networks to approximate complicated functions and create useful internal representations.

Desirable Properties

A useful activation function should support effective gradient-based learning, have suitable numerical behavior, and avoid unnecessary computational cost. The best choice depends on the network and task.

Activation and Backpropagation

During backpropagation, derivatives of activation functions influence the gradients. Poor gradient behavior can make training slow or unstable.

6. Sigmoid, Tanh and ReLU

Sigmoid

The sigmoid function is $\sigma(z) = 1 / (1 + e^{(-z)})$. Its output lies between 0 and 1. It is useful when an output is naturally interpreted as a probability, especially in binary classification.

Advantages: smooth, bounded and easy to interpret as a probability. **Disadvantages:** it can suffer from vanishing gradients when values become very positive or very negative.

Hyperbolic Tangent (tanh)

The tanh function is $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$. Its output lies between -1 and 1 and is centered around zero.

Tanh can work better than sigmoid in some hidden-layer settings because its outputs are zero-centered, but it can still suffer from vanishing gradients.

Rectified Linear Unit (ReLU)

ReLU is defined as $f(z) = \max(0, z)$. It outputs zero for negative inputs and the input itself for positive values.

Advantages: simple, computationally efficient and generally effective in deep networks. **Disadvantage:** neurons can become inactive for negative inputs, a behavior often called the dying ReLU problem.

7. Advanced Activation Functions

Leaky ReLU

Leaky ReLU keeps a small negative slope instead of producing exactly zero for all negative inputs. A common form is $f(z) = z$ for $z > 0$ and αz for $z \leq 0$, where α is a small positive constant.

Its purpose is to reduce the chance of permanently inactive neurons compared with standard ReLU.

Parametric ReLU (PReLU)

PReLU is similar to Leaky ReLU, but the negative slope is learned during training. This adds a parameter and can improve flexibility.

ELU

Exponential Linear Unit uses a smooth negative region and a linear positive region. It can produce negative outputs and may improve optimization in some settings.

GELU

Gaussian Error Linear Unit, or GELU, smoothly weights inputs according to their magnitude rather than using a hard threshold. It is widely associated with transformer-based architectures.

Softmax

Softmax converts a vector of scores into values that sum to one. For class scores z_i , softmax produces $p_i = \exp(z_i) / \sum_j \exp(z_j)$. It is commonly used in the output layer for multiclass classification.

Swish

Swish is commonly expressed as $f(z) = z \cdot \text{sigmoid}(z)$. It is smooth and has been used in modern deep learning architectures.

8. Comparison of Activation Functions

Functional Comparison

Sigmoid: range 0 to 1; commonly useful for binary-output probabilities; main issue is saturation and vanishing gradients.

Tanh: range -1 to 1 ; zero-centered; useful in some hidden layers but can also saturate.

ReLU: range 0 to infinity; highly popular for hidden layers; efficient but may produce inactive neurons.

Leaky ReLU: allows a small negative output; helps reduce dead-neuron behavior.

GELU: smooth nonlinear activation; widely used in modern transformer-style networks.

Softmax: outputs a probability distribution across multiple classes; normally used at a multiclass classification output.

Choosing an Activation Function

For hidden layers, ReLU and related functions are strong general-purpose choices. For binary classification outputs, sigmoid is often appropriate. For multiclass classification, softmax is commonly used when a normalized class probability distribution is desired.

Important Note

There is no single activation function that is best for every architecture. The choice depends on the problem, architecture, optimization behavior and empirical performance.

1. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press.

2. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer.

3. Steven Nouri and Peter Dooling, *Artificial Intelligence: A Modern Approach*.

4. Michael Nielsen, *Neural Networks and Deep Learning*.

5. Original research literature on ReLU, GELU and related neural network activation functions. Activation functions provide the nonlinear transformations required to build these representations.

Natural Language Processing

Neural networks are used for translation, text classification, summarization, question answering and many other language tasks. Modern architectures often rely on smooth activations such as GELU.

Speech and Audio

Models can learn relationships in audio signals for speech recognition, speaker-related tasks and sound classification.

Healthcare

Machine-learning models can assist with pattern recognition in medical images, risk estimation and analysis of complex biomedical data. Such systems require careful validation and responsible use.

Finance and Security

Neural networks can support fraud detection, anomaly detection, forecasting and classification of suspicious patterns.

Activation Function Considerations

When designing a model, consider gradient behavior, computational cost, numerical stability and the required output interpretation. Activation functions should be selected as part of the overall architecture rather than in isolation.

Common Training Problems

Vanishing gradients, exploding gradients, overfitting, underfitting and poor initialization can affect training. Appropriate activations, normalization, regularization and optimization methods can help.